

DOKUMEN ARSITEKTUR

SAFEALERT

Sistem Alarm Darurat Multisensori

Berbasis Kendali Terpusat

Hackathon 24 Jam - Blueprint Lengkap

18 Mei 2026

1. RINGKASAN KEPUTUSAN BERSAMA

Berikut adalah keputusan arsitektur yang telah disepakati melalui diskusi:

Aspek	Keputusan
Backend	Satu project Django (DRF untuk API mobile + Django Templates/Bootstrap untuk dashboard admin)
Push Notification	Firebase Cloud Messaging (FCM) token per device. Backend simpan semua token di database
Registrasi User	User isi data di app -> masuk status "Menunggu Verifikasi" -> admin approve -> jadi "Aman"
Data User	Nama, No HP, Lantai (manual), Tipe Disabilitas, FCM Token, Admin Status
Status Admin	Menunggu Verifikasi, Aman, Sedang Dievakuasi, Terjebak, Diluar Gedung
Status User (Konfirmasi)	Saya Aman, Saya Terjebak, Sedang Evakuasi
Alarm	Admin ketik pesan bebas -> tekan tombol -> backend kirim FCM ke semua token terdaftar -> HP us
Personalized Alert	Beda per tipe disabilitas (switch case di Flutter). Backend kirim notifikasi umum saja
Riwayat Alarm	Wajib ada - tabel Alarm_Log menyimpan pesan, waktu, admin, jumlah penerima, status
Konfirmasi User	Wajib ada - user pilih status laporan saat alarm. Dashboard tampilkan perbandingan Admin Status
Telepon	Tombol "Hubungi 113/110" di app user (buka dialer HP). Nomor HP user di dashboard bisa diklik (t
Token mati/uninstall	Skip (nice to have)
GPS otomatis / deteksi lantai	Skip (nice to have)
SOS dua arah / fallback SMS	Skip (nice to have)
Platform	Android only (iOS critical alert butuh izin Apple, terlalu lama)

2. ENTITY RELATIONSHIP DIAGRAM (ERD)

2.1 Diagram Relasi



2.2 Tabel User

Kolom	Tipe	Keterangan
id	Integer PK	Auto increment
name	Varchar(100)	Nama lengkap user
phone	Varchar(20)	Nomor HP (unik)
floor	Integer	Lantai tempat user berada (input manual)
disability_type	Varchar(20)	deaf / blind / none
fcm_token	Text	Token Firebase Cloud Messaging per device
admin_status	Varchar(20)	pending / safe / evacuating / trapped / outside
created_at	DateTime	Timestamp registrasi

2.3 Tabel Admin

Kolom	Tipe	Keterangan
id	Integer PK	Auto increment
username	Varchar(50)	Username login
password	Varchar(255)	Hash password (Django default)
created_at	DateTime	Timestamp

2.4 Tabel Alarm_Log

Kolom	Tipe	Keterangan
id	Integer PK	Auto increment
message	Text	Pesan alarm yang diketik admin
triggered_by	Integer FK	ID admin yang memicu alarm
triggered_at	DateTime	Waktu alarm dikirim
total_recipients	Integer	Jumlah user yang menerima notifikasi

status	Varchar(20)	active / cancelled
--------	-------------	--------------------

2.5 Tabel Confirmation

Kolom	Tipe	Keterangan
id	Integer PK	Auto increment
user_id	Integer FK	ID user yang konfirmasi
alarm_id	Integer FK	ID alarm yang dikonfirmasi
confirmed_at	DateTime	Waktu konfirmasi
user_reported_status	Varchar(20)	safe / trapped / evacuating

3. ALUR DATA END-TO-END

3.1 Registrasi User (Pertama Kali Install App)

```
[User Buka App Flutter]
|
[Isi Form: Nama, No HP, Lantai, Tipe Disabilitas]
|
[HP generate FCM Token via Firebase SDK]
|
[POST /api/register/ -> Backend Django]
|
[Backend simpan ke tabel User]
  admin_status = pending
|
[Response: Registrasi berhasil. Menunggu verifikasi admin.]
|
[App Flutter simpan user_id dan token lokal (SharedPreferences)]
```

3.2 Admin Verifikasi User (Dashboard Web)

```
[Admin Login Web Dashboard]
|
[Tab Verifikasi User - tabel user dengan admin_status = pending]
|
[Admin klik tombol Verifikasi atau Tolak]
|
[PATCH /api/users/{id}/status/ -> body: {admin_status: safe}]
|
[User sekarang aktif dan bisa menerima alarm]
```

3.3 Admin Trigger Alarm (Inti Sistem)

```
[Admin terima laporan dari satpam/call center]
|
[Buka Dashboard -> Tab Alarm]
|
[Ketik pesan bebas di textarea]
  Contoh: KEBAKARAN LT. 7. GUNAKAN TANGGA DARURAT TIMUR.
|
[Klik tombol merah KIRIM ALARM KE SEMUA USER]
|
[Backend proses:]
  1. Buat row Alarm_Log (status = active)
  2. Ambil semua User dengan admin_status IN (safe, evacuating, trapped)
  3. Hitung jumlah = total_recipients
  4. Kirim FCM ke semua token (multicast/batch)
    Payload: {alarm_id: 123, message: ..., type: emergency}
```

```
5. Update Alarm_Log.total_recipients
|
[Firestore kirim ke HP user]
|
[HP user -> force fullscreen alert]
```

3.4 User Konfirmasi Alarm (Dua Arah)

```
[Alarm fullscreen muncul di HP user]
|
[User baca instruksi evakuasi]
|
[User pilih salah satu tombol:]
  [SAYA AMAN] [SAYA TERJEBAK] [SEDANG EVAKUASI]
|
[App kirim POST /api/confirm/]
  Body: {
    alarm_id: 123,
    user_id: 456,
    user_reported_status: safe
  }
|
[Backend buat row Confirmation]
|
[Dashboard admin auto-refresh]
  Tampilkan: 12 dari 50 user sudah konfirmasi
```

3.5 Admin Update Status User (Manual)

```
[Admin lihat daftar user di dashboard]
|
[Klik dropdown status user tertentu]
|
[Pilih: Sedang Dievakuasi / Terjebak / Diluar Gedung / Aman]
|
[PATCH /api/users/{id}/status/]
|
[Update User.admin_status di database]
```

3.6 Admin Batalkan Alarm (Opsional)

```
[Admin klik tombol BATALKAN ALARM di riwayat alarm yang masih active]
|
[Backend kirim FCM notifikasi baru: {type: cancel, alarm_id: 123}]
|
[HP user terima notifikasi cancel -> stop getar + flash + TTS]
|
[Tampilkan layar hijau: AMAN. KEMBALI BEKERJA.]
|
[Update Alarm_Log.status = cancelled]
```

4. SPESIFIKASI API ENDPOINT

4.1 POST `/api/auth/register/` (User Mobile)

Request:

```
{
  "name": "Budi Santoso",
  "phone": "081234567890",
  "floor": 7,
  "disability_type": "deaf",
  "fcm_token": "dI9pL7..."
}
```

Response 201:

```
{
  "id": 1,
  "name": "Budi Santoso",
  "phone": "081234567890",
  "floor": 7,
  "disability_type": "deaf",
  "admin_status": "pending",
  "message": "Registrasi berhasil. Menunggu verifikasi admin."
}
```

4.2 POST `/api/auth/login/` (Admin Dashboard)

Request:

```
{
  "username": "admin_gedung",
  "password": "password123"
}
```

Response 200:

```
{
  "token": "jwt_token_here",
  "admin": {
    "id": 1,
    "username": "admin_gedung"
  }
}
```

4.3 GET `/api/users/`

Headers: Authorization: Bearer <token>

Response 200:

```
{
  "count": 50,
  "results": [
    {
      "id": 1,
      "name": "Budi Santoso",
      "phone": "081234567890",
      "floor": 7,
      "disability_type": "deaf",
      "admin_status": "safe",
      "created_at": "2026-05-18T10:00:00Z"
    }
  ]
}
```

4.4 PATCH /api/users/{id}/status/

Headers: Authorization: Bearer <token>

Request:

```
{
  "admin_status": "safe"
}
```

Response 200:

```
{
  "id": 1,
  "admin_status": "safe",
  "message": "Status user diperbarui."
}
```

4.5 POST /api/alarms/trigger/

Headers: Authorization: Bearer <token>

Request:

```
{
  "message": "KEBAKARAN LT. 7. GUNAKAN TANGGA DARURAT TIMUR."
}
```

Response 201:

```
{
  "alarm_id": 123,
  "message": "KEBAKARAN LT. 7...",
  "triggered_at": "2026-05-18T14:32:00Z",
}
```



```
"total_recipients": 50,  
"status": "active"  
}
```

4.6 GET /api/alarms/

Headers: Authorization: Bearer <token>

Response 200:

```
{  
  "count": 10,  
  "results": [  
    {  
      "id": 123,  
      "message": "KEBAKARAN LT. 7...",  
      "triggered_by": "Pak Budi",  
      "triggered_at": "2026-05-18T14:32:00Z",  
      "total_recipients": 50,  
      "confirmed_count": 12,  
      "status": "active"  
    }  
  ]  
}
```

4.7 POST /api/alarms/{id}/cancel/

Headers: Authorization: Bearer <token>

Response 200:

```
{  
  "alarm_id": 123,  
  "status": "cancelled",  
  "message": "Alarm berhasil dibatalkan. Notifikasi aman dikirim ke semua user."  
}
```

4.8 POST /api/confirm/

Request:

```
{  
  "alarm_id": 123,  
  "user_id": 456,  
  "user_reported_status": "safe"  
}
```

Response 201:

```
{
```

```
{
  "id": 789,
  "alarm_id": 123,
  "user_id": 456,
  "user_reported_status": "safe",
  "confirmed_at": "2026-05-18T14:35:22Z",
  "message": "Konfirmasi berhasil dicatat."
}
```

4.9 GET /api/alarms/{id}/confirmations/

Response 200:

```
{
  "alarm_id": 123,
  "total_users": 50,
  "confirmed_count": 12,
  "confirmations": [
    {
      "user_id": 456,
      "user_name": "Budi Santoso",
      "user_reported_status": "safe",
      "confirmed_at": "2026-05-18T14:35:22Z"
    }
  ]
}
```

5. SPESIFIKASI DASHBOARD ADMIN (Web)

5.1 Halaman Login

- Form login: Username, Password
- Redirect ke dashboard setelah login

5.2 Layout Dashboard

Sidebar Menu:

- 1. Verifikasi User
- 2. Daftar User
- 3. Alarm & Riwayat

5.3 Tab Verifikasi User

- Tabel user dengan admin_status = pending
- Kolom: Nama, No HP, Lantai, Tipe Disabilitas, Waktu Registrasi
- Aksi: Tombol Verifikasi (hijau) dan Tolak (merah)

5.4 Tab Daftar User

- Tabel semua user yang sudah diverifikasi
- Kolom: Nama, No HP, Lantai, Tipe Disabilitas, Admin Status (dropdown editable), Konfirmasi Terakhir
- Admin Status dropdown: safe, evacuating, trapped, outside
- Nomor HP bisa diklik -> tel:+628... (buka aplikasi telepon)

5.5 Tab Alarm & Riwayat

Bagian Atas - Kirim Alarm Baru:

- Textarea (placeholder: Ketik instruksi evakuasi...)
- Tombol merah besar: KIRIM ALARM KE SEMUA USER
- Tombol disabled kalau textarea kosong

Bagian Bawah - Riwayat Alarm:

Waktu	Pesan	Admin	Total	Konfirmasi	Status	Aksi
14:32	Kebakaran LT 7	Pak Budi	50	12	Aktif	[Batalan]
09:15	Gempa Bumi	Pak Budi	50	48	Selesai	-

Klik baris riwayat -> expand detail daftar user:

- Hijau = sudah konfirmasi + status laporan user
- Kuning = belum konfirmasi
- Tampilkan perbandingan: Admin: Terjebak | User: Saya Aman

6. SPESIFIKASI APLIKASI MOBILE (Flutter)

6.1 Screen 1: Registrasi (Sekali seumur hidup)

- Input: Nama, No HP, Lantai (number picker), Tipe Disabilitas (radio button)
- Tombol Daftar
- Setelah sukses -> simpan user_id & fcm_token di SharedPreferences
- App tidak perlu login lagi setelahnya

6.2 Screen 2: Standby (Home)

- Tampilan sederhana: SafeAlert Aktif
- Tombol Hubungi 113 (Pemadam Kebakaran) -> url_launcher ke tel:113
- Tombol Hubungi 110 (Polisi) -> url_launcher ke tel:110
- Informasi: Nama user, lantai, status verifikasi
- App diam di background, siap terima FCM

6.3 Screen 3: Alarm Fullscreen (Force Alert)

Trigger: Notifikasi FCM dengan type: emergency masuk

Layout:

- Background merah solid (high contrast)
- Teks besar putih (font size 32sp+)
- Isi teks = pesan dari admin
- Getar keras: pola SOS [500, 1000, 500, 1000, 500, 1000] berulang 3 kali
- Flash blink: merah-putih cepat (via torch_light)

Personalized Alert (Switch Case):

```
switch(disabilityType) {  
  case 'deaf':  
    // Getar + Flash + Teks besar  
    // NO suara/TTS  
    break;  
  case 'blind':  
    // TTS baca pesan + Getar  
    // NO flash/visual  
    break;  
  case 'none':  
    // Semua modalitas: Getar + Flash + TTS + Teks  
    break;  
}
```

Tombol Konfirmasi (3 pilihan):

[SAYA AMAN] [SAYA TERJEBAK] [SEDANG EVAKUASI]

Tap salah satu -> kirim POST /api/confirm/ -> dismiss layar alarm

6.4 Screen 4: Notifikasi Batal (Optional)

Trigger: Notifikasi FCM dengan type: cancel

- Background hijau
- Teks: AMAN. KEMBALI BEKERJA.
- Stop semua getar + flash + suara
- Auto-dismiss setelah 5 detik

6.5 Permission Android

INTERNET
VIBRATE
FLASHLIGHT
WAKE_LOCK (supaya layar nyala saat alarm)
POST_NOTIFICATIONS (Android 13+)
FOREGROUND_SERVICE (opsional)

6.6 Firebase Setup

- Buat project Firebase
- Tambahkan app Android (package name: com.safealert.app)
- Download google-services.json -> taruh di android/app/
- Enable Firebase Cloud Messaging

7. SPESIFIKASI PUSH NOTIFICATION (FCM)

7.1 Payload FCM - Alarm Darurat

```
{
  "to": "<device_fcm_token>",
  "priority": "high",
  "data": {
    "type": "emergency",
    "alarm_id": 123,
    "message": "KEBAKARAN LT. 7. GUNAKAN TANGGA DARURAT TIMUR."
  },
  "notification": {
    "title": "ALARM DARURAT",
    "body": "KEBAKARAN LT. 7. SEGERA EVAKUASI!"
  }
}
```

7.2 Payload FCM - Pembatalan

```
{
  "to": "<device_fcm_token>",
  "priority": "high",
  "data": {
    "type": "cancel",
    "alarm_id": 123,
    "message": "AMAN. KEMBALI BEKERJA."
  }
}
```

7.3 Full Screen Intent (Android)

Di payload data, Flutter akan menangkap notifikasi via `firebase_messaging` background handler. Handler ini akan:

- 1. Parse payload
- 2. Buka activity fullscreen menggunakan `flutter_local_notifications` dengan `fullScreenIntent: true`
- 3. Layar langsung nyala meski HP di kantong / DND mode

8. PEMBAGIAN KERJA TIM

Role	Tugas	Stack
Dev 1 (Mobile)	Flutter app + hardware native + FCM	Flutter, Firebase SDK, flutter_tts, torch_light
Dev 2 (Backend)	Django + DRF + database + FCM push	Django, DRF, PostgreSQL, pyfcm
Dev 3 (Dashboard)	Web admin untuk operator	Django Templates, Bootstrap 5

8.1 Fase 1: Setup & Alignment (Semua)

- Dev 2: setup repo GitHub, buat 3 branch (mobile, backend, dashboard)
- Dev 1: install Flutter, setup Firebase project, download google-services.json
- Dev 2: install Django + DRF + PostgreSQL/SQLite, setup virtualenv
- Dev 3: setup Django Templates project, Bootstrap CDN
- Semua: finalisasi ERD dan endpoint API
- Dev 2: push struktur project Django ke GitHub

8.2 Fase 2: Backend Foundation (Dev 2)

- Target: API dasar siap di-test via Postman
- Buat model User, Admin, Alarm_Log, Confirmation
- Setup Django REST Framework + SimpleJWT auth
- Endpoint register user, login admin, get/patch user status
- Setup firebase-admin-python untuk kirim FCM
- Push ke GitHub

8.3 Fase 3: Mobile UI + Hardware (Dev 1)

- Target: App bisa registrasi + terima notifikasi + multisensory alert
- Screen Registrasi (form + simpan ke SharedPreferences)
- Integrasi firebase_messaging - setup background handler
- Implementasi force fullscreen alarm (getar + flash + TTS)
- Personalized alert switch case (deaf/blind/none)
- Tombol konfirmasi 3 pilihan + tombol telepon 113 & 110
- Push ke GitHub

8.4 Fase 4: Dashboard UI (Dev 3)

- Target: Operator bisa login, verifikasi user, lihat daftar, trigger alarm
- Halaman Login + Layout Sidebar + Main Content
- Tab Verifikasi User: tabel pending + tombol verifikasi/tolak

- Tab Daftar User: tabel aktif + dropdown edit status + link tel:
- Tab Alarm: textarea pesan + tombol KIRIM ALARM + tabel riwayat + detail konfirmasi
- Integrasi dengan API backend
- Push ke GitHub

8.5 Fase 5: Integration

- Target: End-to-end flow dasar jalan
- Mobile consume API register, simpan token JWT
- Dashboard consume API user list & alarm trigger
- Test: Dashboard tekan Broadcast -> Backend kirim FCM -> Mobile terima fullscreen alarm
- Mobile kirim konfirmasi -> Backend catat -> Dashboard tampilkan
- Dashboard tambah tombol Batalkan Alarm -> kirim FCM cancel

8.6 Fase 6: Polish & Testing

- Tambah screen Standby dengan tombol telepon 113 & 110
- Pastikan nomor HP di dashboard bisa diklik
- Tambah endpoint cancel alarm + kirim FCM cancel
- Handle notifikasi cancel -> layar hijau AMAN
- Tambah indikator visual perbandingan Admin Status vs User Status
- Scenario test lengkap (lihat Bab 9)
- Validasi sederhana (phone unik, floor integer)
- Alert/error handling di dashboard
- Handle permission denied (vibration, flash)

8.7 Fase 7: Deploy & Demo Prep

- Deploy Django ke Railway atau Render
- Deploy dashboard (ikut di Railway/Django Templates)
- Build APK debug (flutter build apk --debug)
- Buat skrip demo 3 menit
- Charge semua device, siapkan hotspot sendiri
- Push final code, tag v1.0-hackathon

9. SCENARIO TEST LENGKAP

Skenario	Aksi	Hasil yang Diharapkan
Registrasi User	User isi form	Data masuk ke backend, status pending
Verifikasi Admin	Admin klik verifikasi	User status jadi safe, muncul di daftar
Kebakaran Lantai 7	Admin tekan Broadcast	Semua ponsel fullscreen alarm
User Tunarungu	User terima alarm	Getar + flash + teks. Tidak ada suara
User Tunanetra	User terima alarm	TTS + getar. Tidak ada flash
User Non-disabilitas	User terima alarm	Semua modalitas
User Konfirmasi	User pilih Saya Aman	Dashboard update, angka konfirmasi naik
Admin Update Status	Admin tandai Terjebak	Dashboard tampilkan perbedaan Admin vs User
False Alarm	Admin batalkan alarm	Notifikasi AMAN terkirim, alarm berhenti
Telepon Darurat	User tekan 113	Buka dialer HP ke 113

10. DELIVERABLE MVP (MINIMUM VIABLE PRODUCT)

No	Komponen
1	Mobile app registrasi user (nama, HP, lantai, tipe disabilitas)
2	Mobile app force fullscreen alarm (getar + flash + TTS + teks)
3	Mobile app personalized alert per tipe disabilitas
4	Mobile app konfirmasi 3 pilihan (Aman / Terjebak / Evakuasi)
5	Mobile app tombol telepon darurat (113 & 110)
6	Backend API register, login, user management
7	Backend API trigger alarm + kirim FCM ke semua token
8	Backend API konfirmasi user + catat riwayat
9	Backend API cancel alarm
10	Dashboard admin login + verifikasi user
11	Dashboard admin daftar user + update status manual
12	Dashboard admin kirim alarm + riwayat alarm
13	Dashboard admin detail konfirmasi per alarm
14	Dashboard admin link telepon ke nomor user

11. CATATAN PENTING & BATASAN

11.1 Batasan yang Diterima

- Token permanen: Kalau user uninstall lalu install ulang, token lama jadi zombie di database. Backend tetap kirim ke token mati (gagal diam-diam). Untuk produksi nanti perlu token cleanup.
- Lantai manual: User bisa salah isi. Tidak ada validasi otomatis.
- Android only: iOS critical alert butuh izin khusus Apple dan provisioning profile, tidak mungkin dalam 24 jam.
- SQLite acceptable: Kalau setup PostgreSQL memakan waktu >30 menit, ganti ke SQLite dulu. Migration bisa dilakukan nanti.
- FCM Multicast: Saat kirim ke ribuan token, gunakan FCM multicast (kirim array token sekaligus) agar tidak loop satu per satu.
- Human-in-the-Loop: Admin mendapat laporan dari luar (satpam/CCTV/call center), lalu memicu alarm manual. Tidak ada sensor otomatis di sistem.
- No Internet Fallback: Kalau HP user tidak punya internet, notifikasi tidak sampai. SMS fallback (Twilio) masuk nice-to-have.
- Battery: GPS tracking tidak aktif terus-menerus. Hanya aktif saat mode evakuasi (untuk hackathon ini, GPS di-skip sama sekali).

11.2 Teknologi Summary

Layer	Teknologi
Mobile App	Flutter, Dart
Push Notification	Firebase Cloud Messaging (FCM)
Backend API	Django 4.x, Django REST Framework
Database	PostgreSQL (atau SQLite untuk prototyping)
Dashboard Web	Django Templates, Bootstrap 5
Auth	SimpleJWT (JSON Web Token)
Deploy	Railway / Render (gratis)
Version Control	GitHub

Dokumen ini merupakan hasil diskusi arsitektur dan siap dieksekusi dalam batas waktu 24 jam hackathon.

Versi: 1.0 | Tanggal: 18 Mei 2026

Tim: Dev 1 (Mobile) | Dev 2 (Backend) | Dev 3 (Dashboard)